

MARS ROVER MANIPAL

RESEARCH AI TASKPHASE

Report 4

Adarsh Inaganti
240905294
adarshinaganti006@gmail.com

1. Review questions

1.1 Pandas questions

1.1.1 What are the functions of `concat()`, `merge()` and `join()`?

In pandas, `concat()`, `merge()`, and `join()` are all used to combine DataFrames, but they work differently.

1. `concat()`: concatenation

- Used to stack DataFrames vertically (rows) or horizontally (columns).
- Does not require a common column.
- Mainly for simple combinations along an axis.
- `pd.concat([df1, df2], axis=0, ignore_index=False)`

2. `merge()`: database-style join

- Works like SQL JOIN.
- Requires a key column(s) to join on.
- Can perform inner, left, right, outer joins.
- `pd.merge(df1, df2, how='inner', on='key_column')`

3. `join()`: join on index

- Primarily used to join DataFrames on their index.
- Simpler alternative to `merge()` when working with indexes instead of key columns.
- Can also join on a column using `set_index()` first.
- `df1.join(df2, how='left')`

1.2 NumPy questions

1.2.1 What is broadcasting?

In NumPy, broadcasting is a powerful mechanism that allows arrays of different shapes to be combined in arithmetic operations without explicitly reshaping them. It makes vectorized operations flexible and memory-efficient.

- When performing operations on arrays, NumPy automatically “stretches” the smaller array so that their shapes are compatible.

- No actual data is copied; NumPy just reuses the smaller array across the larger one.

Rules of broadcasting

1. Compare array shapes from the trailing dimensions.
2. Two dimensions are compatible if:
 - They are **equal**, or
 - One of them is **1**.
3. If shapes are not compatible, NumPy raises a ValueError.

Example 1: adding a scalar to an array

```
import numpy as np
arr = np.array([1, 2, 3])
result = arr + 5
print(result)
```

Output: [6 7 8]

Example 2: adding arrays of different shapes

```
a = np.array([[1,2,3],
               [4,5,6]]) # shape (2,3)
b = np.array([10,20,30]) # shape (3,)
result = a + b
print(result)
```

Output: [[11 22 33]
[14 25 36]]

Example 3: adding a column vector to a matrix

```
c = np.array([[1],[2]]) # shape (2,1)
d = np.array([[10,20,30]]) # shape (1,3)
result = c + d
print(result)
```

Output: [[11 21 31]
[12 22 32]]

Why broadcasting is useful

- Enables vectorized operations without manual reshaping or loops.

- Saves memory, as arrays are not physically copied.
- Works for arithmetic, logical, and comparison operations.

1.3 Git questions

1.3.1 How does Git differentiate commits?

Git differentiates commits using a unique identifier called a SHA-1 hash.

1. Each commit is unique

- When a commit is created, Git generates a 40-character SHA-1 hash (a long hexadecimal string, e.g., a1b2c3d4 . . .).
- This hash acts as the commit ID, uniquely identifying the commit in the repository.

2. Metadata of each commit

- Git calculates the hash based on:
 - The content of the files staged for commit.
 - Commit metadata:
 - Author name & email
 - Commit message
 - Timestamp
 - Parent commit(s) hash
- Even if only one character changes in a file or a different commit message is used, Git generates a completely different hash.

3. How Git tracks history

- Commits are linked to their parent commit via the parent hash.
- This forms a commit tree or DAG (Directed Acyclic Graph).
 - A <- B <- C
 - C points to B as parent, B points to A.
- Git uses hashes, not sequential numbers, so commit IDs are independent of order.

4. Other identifiers

- Branches and tags point to commit hashes, not the content itself.
- Git always uses the hash to check if two commits are the same or different, even across clones.

5. Why Git uses SHA-1

- SHA-1 produces a practically unique fingerprint of content.
- Guarantees integrity: changing a single character changes the hash drastically.

1.3.2 What are SHA keys?

SHA keys (or SHA hashes) are cryptographic identifiers used to uniquely represent data. In Git and many other applications, they are fundamental for integrity and security.

1. What is SHA?

- SHA stands for Secure Hash Algorithm.
- It's a hash function: it takes any input (file, string, etc.) and produces a fixed-length string of characters, usually hexadecimal.
- Even a tiny change in the input produces a completely different hash.
- **Common SHA types:**
 - SHA-1
 - SHA-256
 - SHA-512

2. SHA in Git

- Each commit, blob, tree, and tag in Git is identified by a SHA-1 hash.
- **Why Git uses SHA:**
 - **Uniqueness:** Each commit has a practically unique fingerprint.
 - **Integrity:** If files or metadata are altered, the SHA changes, so Git can detect tampering.
 - **History tracking:** Commits link to parent commits via SHA, forming a DAG (Directed Acyclic Graph).

3. Properties of SHA

- **Deterministic:** Same input gives you the same hash.
- **Fast to compute:** Even large files can be hashed quickly.
- **Irreversible:** You cannot reconstruct original data from SHA (one-way).
- **Avalanche effect:** Tiny change leads to a completely different hash.

1.3.3 What are the differences between fetching, pulling, cloning and forking?

In Git, `fetch`, `pull`, `clone`, and `fork` are all ways to get data from a repository, but they work differently.

1. `git clone`: copy a repository

- **Purpose:** Create a local copy of a remote repository for the first time.
- Downloads all files, branches, commits, and history.
- Creates a new directory with a `.git` folder.
- `git clone https://github.com/user/repo.git`

2. `git fetch`: download changes

- **Purpose:** Update your local copy of remote branches without merging them.
- Downloads new commits, branches, and tags from remote.
- **Does not** change your working files automatically.
- `git fetch origin`

3. `git pull`: fetch and merge

- **Purpose:** Update your local branch with remote changes and merge them immediately.
- `git pull = git fetch + git merge.`
- `git pull origin main`

4. Fork: copy a repository on GitHub/GitLab

- **Purpose:** Create your own remote copy of someone else's repository on GitHub/GitLab.
- Done on the server, not via Git commands.
- You can freely push changes to your fork without affecting the original repo.
- **Workflow Example:**
 - Fork repo on GitHub, this creates `your-username/repo`.
 - Clone your fork locally using `git clone`.
 - Make changes, commit, push to your fork.
 - Submit a pull request to merge into the original repo.

1.3.4 What is the function of `git log`? What are the other arguments one can add to it?

`git log` is used to view the commit history of a repository. It shows commits in reverse chronological order (most recent first) along with useful details like the commit hash, author, date, and message.

Common arguments of `git log`

Argument	Description
<code>--oneline</code>	shows each commit as a single line (short hash + message).
<code>--graph</code>	shows a text-based graph of branches and merges.
<code>--decorate</code>	shows branch names, tags, and refs along with commits.
<code>--author="name"</code>	filters commits by author.
<code>--since="yyyy-dd-mm"</code> <code>--until="yyyy-dd-mm"</code>	show commits after/before a certain date.
<code>-p</code> <code>--patch</code>	shows the diff introduced by each commit.
<code>--stat</code>	shows summary of changes (lines added/removed per file).
<code>-n <number></code>	limits output to last n commits.
<code>--reverse</code>	shows commits in chronological order (oldest first).
<code>--grep="keyword"</code>	filters commits whose messages match the keyword.
<code>--abbrev-commit</code>	shortens commit hashes (default in <code>--oneline</code>).

1.3.5 How are two branches merged?

Merging is the process of combining the changes from one branch into another.

Suppose we have two branches:

- `main` (the branch to merge into)
- `feature` (the branch with changes)

1. Basic merge

- Switch to target branch (`main`)
`git checkout main`
- Merge the other branch (`feature`) into it
`git merge feature`

2. Fast-forward merge

- If the target branch has no new commits since the branch diverged, Git just moves the pointer forward.
- No merge commit is created.

3. Three-way merge

- If both branches have new commits, Git performs a three-way merge:
 - Uses the common ancestor of the two branches.
 - Combines changes from both branches.
 - If conflicts exist, Git pauses for you to resolve them manually.

4. Handling conflicts

- If both branches change the same lines, a conflict occurs.
- Git marks conflicts in files like this:

```
<<<<<< HEAD
Changes in main branch
=====
Changes in feature branch
>>>>>> feature
```

- You manually edit the file to resolve the conflict, then:

```
git add <file>
git commit
```

5. Merge strategies

- **Recursive (default):** Handles two-parent merges (most common).
- **Fast-forward only:** Only allows fast-forward merges.
- **Squash merge:** Combines all feature branch commits into a single commit in main:

```
git merge --squash feature
```

1.3.6 Why can't merge commits be copied from two different branches?

1. What is a merge commit?

- A merge commit is a commit with two (or more) parent commits.
- It represents the point where two branches come together, preserving the history of both.

- Example:

```
A---B---C (main)
      \
        D---E (feature)
```

- If you merge `feature` into `main`, Git creates a new merge commit `M` with parents `C` and `E`.

```
A---B---C---M (main)
      \      /
        D---E (feature)
```

2. Why merge commits can't be copied

- **Parent commits are fixed**
 - Each commit in Git contains hashes of its parent(s).
 - A merge commit references exact commits from both branches.

- If you try to “copy” the merge commit to another branch where the parents don’t match, the commit hash would change, because the parents are different.
- **Merge commits depend on branch history**
 - The commit represents a merge of the specific histories of the two branches at that point in time.
 - Copying it elsewhere would break this integrity of history.
- **Git’s history is immutable**
 - Git hashes are content + metadata + parent hashes.
 - Changing parents (i.e., merging onto a different branch) changes the commit hash, so it is no longer the same commit.

3. How Git handles this

If you want to bring the same changes into another branch, Git offers:

- **Merge again:** creates a new merge commit with the correct parents.
- **Cherry-pick:** applies only the changes, creates a new commit on the target branch.
- **Example:**

```
# you have branch 'feature' merged into 'main'
# now you want it in 'develop' too
git checkout develop
git merge feature # creates a new merge commit for develop
```
- You cannot reuse the merge commit M from main, because its parents include commits from main, which don’t exist in develop’s history.

1.3.7 What are the different types of merges and how are they resolved?

1. Fast-forward merge

- **When it happens:** The target branch has no new commits since the feature branch diverged.
- **How it works:** Git simply moves the target branch pointer forward to the latest commit of the feature branch.
- No merge commit is created.

```
main:      A---B
feature:    C---D
```

```
git checkout main
git merge feature
```

Result:

```
main: A---B---C---D
```

Resolution: Automatic, no conflicts, no extra commit.

2. Three-way merge (recursive merge)

- **When it happens:** Both branches have new commits since they diverged.
- **How it works:** Git finds the common ancestor of the two branches and merges the changes.
- A merge commit is created to tie the histories together.

```
main:      A---B---C
feature:    B---D---E
```

```
git checkout main
git merge feature
```

```
Result:
A---B---C---M (main)
      \      /
      D---E (feature)
```

- M is the merge commit.

Resolution:

- If the changes affect different lines/files, Git merges automatically.
- If the changes conflict, Git marks the file for manual conflict resolution.

3. Squash merge

- **Purpose:** Combine all commits from a feature branch into a single commit on the target branch.

- **How it works:**

```
git merge --squash feature
git commit
```

Example: feature branch has 5 commits. After squash merge, 1 commit added to main.

- **Resolution:** Conflicts handled during squash; afterward, a single commit is created.

4. Octopus merge

- **Purpose:** Merge more than two branches at once.
- **Use case:** Usually for integration branches.

```
git merge branch1 branch2 branch3
```

Resolution:

- Works only if no complex conflicts exist across branches.
- If conflicts are complex, Git refuses and asks to merge branches individually.

5. **Rebase (alternative to merge)**

- Not a true merge, but often used to integrate changes linearly.
- **How it works:** Moves feature branch commits on top of the target branch.

```
git checkout feature  
git rebase main
```
- **Advantages:** Cleaner history, no merge commits.
- **Resolution:** Conflicts must be manually resolved during rebase.

6. **Conflict resolution**

Git marks conflicts like this:

```
<<<<<<< HEAD  
Changes in main branch  
=====  
Changes in feature branch  
>>>>>>> feature
```

Steps:

- Edit the file to resolve conflicts.
- Stage the file: `git add <file>`.
- Complete the merge: `git commit` (if not automatic).

1.3.8 **How do you use `git reset` and `git revert`? Which is used where?**

`git reset` and `git revert` are both used to undo changes, but they work differently and are used in different scenarios.

1. **`git reset`: move the HEAD**

- **Purpose:** Undo commits by moving the current branch pointer (HEAD) to a previous commit.
- Affects history, so can rewrite commit history. Use carefully on shared branches.

```
git reset [--soft | --mixed | --hard] <commit>
```
- **Types of reset:**

Type	Function	Use case
<code>--soft</code>	moves HEAD to <commit> but keeps changes staged	undo commit but keep staged for editing
<code>--mixed</code> (default)	moves HEAD and unstages changes (keeps in working dir)	undo commit, edit before re-committing
<code>--hard</code>	moves HEAD and discards all changes	permanently remove commits and changes

2. **git revert: create an “undo” commit**

- **Purpose:** Undo a commit by creating a new commit that reverses the changes.
- **Does not** rewrite history, safe for shared/public branches.

```
git revert <commit>
```

3. **Which is used where**

`git reset` is used for local undo of commits before pushing and for rewriting commit history.

`git revert` is used to undo commits safely on public/shared branches.

1.4 Machine learning questions

1.4.1 What are continuous and discrete/discontinuous variables?

In statistics, variables are broadly categorized into two main types: continuous and discrete. The term "discontinuous variable" is another name for a discrete variable.

1. Continuous variables

A continuous variable is a variable that can take on any value within a given range. These variables can be broken down into smaller and smaller units, theoretically to an infinite degree.

Key characteristics:

- **Infinite possibilities:** Between any two values of a continuous variable, there can be an infinite number of other values.
- **Measured:** A tool is typically used to measure continuous variables.
- **Represented by:** Real numbers, including fractions and decimals.

Examples:

- Height
- Weight
- Temperature
- Time

2. Discrete variables

A discrete variable, also known as a discontinuous variable, is a variable that can only take on specific, distinct values. There are clear gaps between the possible values.

Key characteristics:

- **Finite or countable values:** The possible values are separate and can be counted.
- **Counted:** You can count the individual units of a discrete variable.
- **Represented by:** Integers (whole numbers).

Examples:

- **Number of students in a class:** You can have 20 students or 21 students, but not 20.5 students.
- **The outcome of a dice roll:** The possible values are 1, 2, 3, 4, 5, or 6.
- **Number of cars in a parking lot:** You count the number of cars.
- **Shoe size:** While shoe sizes can include halves (like 8.5), you can't have a shoe size of 8.25.

1.4.2 What are the assumptions we assume for the dataset used in linear regression?

1. **Linearity:** The most fundamental assumption is that a linear relationship exists between the independent variable(s) and the dependent variable. This means that a straight line can effectively model the relationship between the variables.
2. **Independence of residuals:** The residuals (the differences between the observed and predicted values) should be independent of each other. This means that the residual of one observation should not be predictable from the residual of another.
3. **Homoscedasticity:** This assumption states that the variance of the residuals should be constant at every level of the independent variable(s). In simpler terms, the spread of the residuals should be roughly the same across the entire range of your data.
4. **Normality of residuals:** The residuals of the model should be normally distributed. This doesn't mean your variables themselves need to be normally distributed, but the errors in the predictions of the model should be.
5. **No multicollinearity:** In multiple linear regression (with more than one independent variable), the independent variables should not be highly correlated with each other.

1.4.3 What is a hypothesis?

In the context of linear regression, the null and alternative hypotheses are used to test the statistical significance of the relationship between the independent and dependent variables.

1. Null hypothesis (H_0)

The null hypothesis in linear regression states that there is no linear relationship between the independent variable(s) and the dependent variable. In other words, it proposes that the independent variable has no effect on the dependent variable. This is typically expressed by stating that the coefficient (slope) of the independent variable is equal to zero.

$$H_0: w = 0$$

2. Alternative hypothesis (H_a)

The alternative hypothesis is the opposite of the null hypothesis. It states that there is a linear relationship between the independent variable(s) and the dependent variable. This is expressed by stating that the coefficient (slope) of the independent variable is not equal to zero.

$$H_a: w \neq 0$$

1.4.4 What is correlation and what are its types?

Correlation is a statistical measure that expresses the extent to which two variables are linearly related, meaning they change together at a constant rate. It is a common tool for describing simple relationships without making a statement about cause and effect.

The strength of a correlation is measured by the correlation coefficient, which is typically represented by the symbol $\mathbf{r}$. The value of $\mathbf{r}$ ranges from -1 to +1:

- A value close to **+1** indicates a strong positive correlation.
- A value close to **-1** indicates a strong negative correlation.
- A value close to **0** indicates a weak or no linear correlation.

Types of correlation

1. **Positive correlation:** A positive correlation exists when two variables move in the same direction. As one variable increases, the other variable also increases, and as one decreases, the other also decreases.
2. **Negative correlation:** A negative correlation exists when two variables move in opposite directions. As one variable increases, the other variable decreases, and vice versa.
3. **No correlation (or zero correlation):** No correlation (or zero correlation) exists when there is no apparent relationship between two variables. A change in one variable does not predict a change in the other.

1.4.5 What is a correlation matrix?

- A correlation matrix is a table that shows the correlation coefficients between several variables. It is a powerful tool for summarizing the relationships within a set of data in a concise and easy-to-read format.
- A correlation matrix is always a square matrix, meaning it has the same number of rows and columns.
- Each row and each column represents a different variable. The cell where a row and column intersect shows the correlation between those two variables.
- A key feature of a correlation matrix is that the diagonal from the top-left to the bottom-right always consists of 1s. This is because a variable is always perfectly correlated with itself.
- The values in the matrix are correlation coefficients, which range from -1 to +1.
 - **+1:** A perfect positive correlation.
 - **-1:** A perfect negative correlation.
 - **0:** No linear correlation.
- The matrix is also symmetrical, meaning the correlation between Variable A and Variable B is the same as the correlation between Variable B and Variable A.

1.4.6 What is multicollinearity?

Multicollinearity is a statistical issue that occurs in a regression model when two or more independent variables are highly correlated with each other. This high level of correlation means that the variables are not truly independent and can interfere with each other's effects in the model.

Why multicollinearity is a problem

When independent variables are strongly correlated, it becomes difficult for the model to distinguish their individual impacts on the dependent variable. This leads to several problems:

- ***Unreliable coefficients:*** The estimated coefficients of the correlated variables can become very sensitive to small changes in the data. They might have large standard errors, making them statistically insignificant even when a relationship exists.
- ***Difficulty in interpretation:*** It becomes challenging to determine the effect of each independent variable. For example, if advertising spending and marketing team size are highly correlated, it's hard to say which one is truly driving sales.

- **Model instability:** The coefficients can swing wildly, even changing signs (from positive to negative), depending on which variables are included in the model.

How to detect multicollinearity

The most common way to detect multicollinearity is by using the Variance Inflation Factor (VIF). The VIF measures how much the variance of an estimated regression coefficient is increased because of multicollinearity.

- **VIF = 1:** No correlation.
- **VIF > 1 and < 5:** Moderate correlation, but often not a cause for concern.
- **VIF > 5 or 10:** High correlation, indicating a potential problem with multicollinearity.

1.4.7 Why is MSE (mean squared error) used instead of MAE (mean absolute error)?

While both Mean Squared Error (MSE) and Mean Absolute Error (MAE) are common metrics for measuring the accuracy of a model, MSE is often preferred because it penalizes larger errors more severely than smaller ones.

- **MAE (Mean Absolute Error):** Takes the absolute value of the error. This means that all errors are weighted equally in the average. An error of 10 is treated as being exactly twice as bad as an error of 5.

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

- **MSE (Mean Squared Error):** Squares the error. This results in a much larger penalty for bigger errors. For example, an error of 10 becomes 100, while an error of 5 becomes 25. In this case, the larger error contributes four times as much to the total error.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Another significant advantage of MSE is that it is differentiable, which means it has a smooth curve. This makes it easier to use with optimization algorithms like gradient descent, which are used to find the best-fit line in linear regression. The function for MAE, due to the absolute value, is not differentiable at all points, making optimization more complex.

1.4.8 What is a partial derivative?

A partial derivative is a derivative of a function with several variables, where you find the derivative with respect to one variable while holding all the other variables constant.

Example

$$f(x) = 3x^2 + 4xy - 5y^3$$

- $\frac{\partial f}{\partial x} = 6x + 4y$
- $\frac{\partial f}{\partial y} = 4x - 15y^2$

1.4.9 Why is the partial derivative of the cost function with respect to the features divided by the total number of training samples?

- Dividing the partial derivative of the cost function by the total number of training samples is done to calculate the average gradient. This simple step is crucial for making the learning process stable and consistent.
- The main reason for averaging is to make the learning rate independent of the number of training samples.
- By averaging the gradient over the training samples, one gets a better and more stable estimate of the "true" gradient of the error across the entire data distribution. This helps the model generalize better to new, unseen data.

1.4.10 How is the learning rate tuned?

1. Manual tuning

This is the most basic method and relies on trial and error. You start with a common learning rate (e.g., 0.1, 0.01, or 0.001) and observe the training loss.

- If the loss decreases very slowly, the learning rate might be too small. Try increasing it.
- If the loss plateaus, or starts to increase, the learning rate might be too large. Try decreasing it.

2. Grid search and random search

- **Grid search:** You define a grid of learning rates (e.g., [0.1, 0.01, 0.001, 0.0001]) and train the model with each one. You then select the learning rate that results in the best performance.
- **Random search:** Instead of a predefined grid, you randomly sample learning rates from a given range. This can sometimes be more efficient than grid search, as it explores a wider range of values.

3. Learning rate schedulers

A learning rate scheduler dynamically adjusts the learning rate during training. A common strategy is to start with a relatively high learning rate and then gradually decrease it as training progresses.

- **Step decay:** Reduces the learning rate by a factor at specific intervals (epochs).
- **Exponential decay:** Gradually decreases the learning rate after each epoch.
- **Cosine annealing:** Varies the learning rate following a cosine curve, which can help the model escape local minima.

1.4.11 What are the signs of a high or low learning rate?

1. Signs of a high learning rate

- **Loss fluctuates or explodes:** The most common sign is a loss that bounces around erratically or even increases over time. Instead of a smooth downward trend, the loss curve will look chaotic.
- **Rapid initial decrease, then plateau:** The loss might decrease very quickly for the first few iterations and then suddenly stop improving or start to worsen.
- **Very large weight updates:** The model's weights will change drastically from one update to the next.

2. Signs of a low learning rate

- **Loss decreases very slowly:** The loss will go down, but the progress will be extremely gradual.
- **Takes a long time to converge:** The model will require a very large number of epochs to reach a good solution.
- **Gets stuck in a suboptimal solution:** The small updates might not be enough to push the model out of a local minimum or a plateau in the loss landscape.

An ideal learning rate will show a smooth and steady decrease in the loss function over time. It will be fast enough to converge in a reasonable number of epochs but slow enough to avoid overshooting the optimal solution.

1.4.12 What is R^2 score and adjusted R^2 score?

Both the R-squared (R^2) and Adjusted R-squared are metrics that measure how well a regression model fits the data, but the Adjusted R^2 provides a more accurate assessment, especially when the model has multiple independent variables.

1. R^2 score

- The R-squared score, also known as the coefficient of determination, measures the proportion of the variance in the dependent variable that is predictable from the independent variable(s).
- It tells us what percentage of the variation in the output variable can be explained by the input variable(s).
 - **Range:** 0 to 1 (or 0% to 100%).
 - **Interpretation:** An R^2 of 0.82 means that 82% of the variation in the dependent variable can be explained by the model.
- The major flaw of R^2 is that it almost always increases as you add more independent variables to the model, even if those variables are not useful for predicting the outcome.

2. Adjusted R^2 score

- The Adjusted R-squared is a modified version of R^2 that has been adjusted for the number of predictors in the model.
- Unlike R^2 , the Adjusted R^2 will only increase if the new variable improves the model more than would be expected by chance.
- This makes it a more reliable metric when you are comparing models with different numbers of independent variables.
 - **Penalty for complexity:** The Adjusted R^2 penalizes the model for having extra variables that are not contributing to the model's explanatory power.
 - **Can decrease:** If you add a useless variable to the model, the Adjusted R^2 can actually decrease.

1.4.13 What is the difference between an iteration and an epoch?

- An epoch is one complete pass through the entire training dataset.
- An iteration is a single update of the model's weights. This happens after the model has processed a batch of data.
- The relationship between them depends on the batch size, which is the number of training samples you use in one iteration.

$$\text{Iterations} = \text{Total training samples} / \text{batch size}$$

1.4.14 What are the different types of gradient descent?

1. Batch gradient descent

Batch Gradient Descent is the most straightforward type. It calculates the gradient of the cost function using the entire training dataset for each update of the model's parameters.

Pros:

- **Stable convergence:** The gradient is calculated over the entire dataset, so the updates are stable and the convergence is smooth.
- **Guaranteed to converge:** It's guaranteed to converge to the global minimum for convex error surfaces and to a local minimum for non-convex surfaces.

Cons:

- **Slow:** It can be very slow and computationally expensive, especially for large datasets.
- **Memory intensive:** Requires the entire dataset to be in memory.

2. Stochastic Gradient Descent (SGD)

Stochastic Gradient Descent (SGD) is a faster alternative to Batch Gradient Descent. Instead of using the entire dataset, it updates the model's parameters after each single training sample.

Pros:

- **Fast:** The frequent updates allow for much faster computation.
- **Can escape local minima:** The noisy updates can help the model jump out of local minima.

Cons:

- **Noisy convergence:** The path to the minimum is very noisy and can be erratic.
- **May not converge:** The updates can be so noisy that the model might not converge to the exact minimum.

3. Mini-batch Gradient Descent

Mini-batch Gradient Descent is a compromise between Batch Gradient Descent and SGD. It updates the model's parameters after processing a small, random batch of training samples. This is the most common type of gradient descent used in deep learning.

Pros:

- **Efficient:** It's more computationally efficient than Batch Gradient Descent.
- **Stable:** The updates are more stable than with SGD.
- **Takes advantage of vectorization:** It can leverage the power of matrix optimizations.

Cons:

- **Requires tuning:** It introduces a new hyperparameter, the batch size, which needs to be tuned.

1.4.15 Why do we need to feature engineer?

- We need to perform feature engineering to improve the performance of machine learning models by transforming raw data into a format that better represents the underlying problem.
- The primary goal of feature engineering is to increase the accuracy and predictive power of your model. By creating features that highlight the important relationships in the data, you make it easier for the model to learn and make accurate predictions.
- Many machine learning algorithms have specific requirements for the format of their input data. Feature engineering is used to transform your data to meet these requirements.
- By selecting only the most important features and representing them efficiently, you can reduce the complexity and dimensionality of your data. This can lead to:
 - Faster training times.
 - Reduced risk of overfitting.
 - Models that are easier to interpret.

1.4.16 Why is logistic regression called “regression” if it’s a classification model?

- Logistic regression is called "regression" because at its core, it predicts a continuous probability, which is a regression task. It becomes a classification model only in the final step when a threshold is applied to that probability.
- The underlying mechanism of logistic regression is very similar to linear regression. It starts with a linear equation to predict a value:

$$z = b + w_1x_1 + w_2x_2 + \dots w_nx_n$$

- However, this value z isn't the final output. Instead, it's passed through a logistic (or sigmoid) function, which squashes the value into a range between 0 and 1.

$$Probability = \frac{1}{1+e^{-z}}$$

- This output is the probability of a given data point belonging to a particular class. Because this probability is a continuous value, this part of the process is a regression task.
- The classification happens after the regression. You set a threshold (typically 0.5) to convert the predicted probability into a class label.
 - If the predicted probability is greater than 0.5, the model classifies the outcome as Class 1.
 - If the predicted probability is less than 0.5, the model classifies the outcome as Class 0.

1.4.17 How do you find the decision boundary in logistic regression?

- In logistic regression, the decision boundary is the line or surface that separates the predicted classes. You find it by setting the linear part of the logistic regression equation to zero.
- The logistic (sigmoid) function gives a probability of 0.5 only when its input, z , is 0:

$$P = \frac{1}{1+e^{-z}} = 0.5 \Rightarrow z = 0$$

- Since z is the linear part of the model, the decision boundary is found precisely where this linear combination equals zero.
- Therefore, the equation for the decision boundary is:

$$b + w_1x_1 + w_2x_2 + \dots w_nx_n = 0$$

1.4.18 What is the loss function in logistic regression?

- The loss function used in logistic regression is called Log Loss, also known as Binary Cross-Entropy Loss.
- This function is designed to measure the performance of a classification model whose output is a probability between 0 and 1. It heavily penalizes the model for being confident and wrong.
- The Log Loss for a single prediction is calculated based on the true label (y) and the predicted probability (p):
 - If the true label is **1 ($y=1$)**: The loss is **$-\log(p)$** .
 - If the model predicts a probability p close to 1 (correct), the loss is close to 0.
 - If the model predicts a probability p close to 0 (incorrect), the loss approaches infinity.
 - If the true label is **0 ($y=0$)**: The loss is **$-\log(1-p)$** .
 - If the model predicts a probability p close to 0 (correct), the loss is close to 0.
 - If the model predicts a probability p close to 1 (incorrect), the loss approaches infinity.
- This behavior ensures that the model is strongly penalized for making confident, incorrect predictions.

1.4.19 What is the difference between loss function and cost function in logistic regression?

In logistic regression, the main difference is the scale: a loss function calculates the error for a single training example, while a cost function is the average error across the entire training dataset.

1. Loss function

- The loss function measures how well the model's prediction matches the true label for a single data point. In logistic regression, this is the Log Loss or Binary Cross-Entropy Loss.
- For one training example with true label y and predicted probability p , the loss is:

$$L = -[y \cdot \log(p) + (1 - y) \cdot \log(1 - p)]$$

2. Cost function

- The cost function gives an overall measure of the model's performance by averaging the loss over all m training examples.
- The goal of training the model is to find the parameters that minimize this cost function.

$$J = \frac{1}{m} \sum_{i=1}^m L_i$$

2. Linear regression

2.1 Introduction and initial setup

The dataset tells us about a list of different cars with their prices and various parameters about each car.

2.1.1 Data dictionary

The data dictionary provided gives us information about the various variables in the dataset.

Car_ID	Unique id of each observation (Integer)
Symboling	Its assigned insurance risk rating. A value of +3 indicates that the auto is risky, -3 that it is probably pretty safe.(Categorical)
carCompany	Name of car company (Categorical)
fueltype	Car fuel type i.e gas or diesel (Categorical)
aspiration	Aspiration used in a car (Categorical)
doornumber	Number of doors in a car (Categorical)
carbody	body of car (Categorical)
drivewheel	type of drive wheel (Categorical)
engineLocation	Location of car engine (Categorical)
wheelbase	Wheelbase of car (Numeric)
carlength	Length of car (Numeric)
carwidth	Width of car (Numeric)
carheight	height of car (Numeric)
curbweight	The weight of a car without occupants or baggage. (Numeric)
enginetype	Type of engine. (Categorical)
cylindernumber	cylinder placed in the car (Categorical)
enginesize	Size of car (Numeric)
fuelsystem	Fuel system of car (Categorical)
boreRatio	Bore ratio of car (Numeric)
stroke	Stroke or volume inside the engine (Numeric)
compressionratio	compression ratio of car (Numeric)
horsepower	Horsepower (Numeric)
peakrpm	car peak rpm (Numeric)
citympg	Mileage in city (Numeric)
highwaympg	Mileage on highway (Numeric)
price(Dependent variable)	Price of car (Numeric)

Here are the different variables along with the respective parameters they represent.

2.2 Data cleaning and preparation

- We can extract the car company into a separate column as this feature would contribute more to our regression model than the car name. This creates a separate `car_company` column.

```
1 # extract car company
2
3 cars['car_company'] = cars['CarName'].apply(lambda x: x.split(' ')[0])
4 cars.head()
```

✓ 0.0s

car_company
alfa-romero
alfa-romero
alfa-romero
audi
audi

- Some car companies have incorrect spellings. This should be corrected as this would result in duplicates.

```
1 # wrong spellings
2
3 corrections = {
4     'maxda': 'mazda',
5     'porcshce': 'porsche',
6     'toyouta': 'toyota',
7     'vokswagen': 'volkswagen',
8     'vw': 'volkswagen'
9 }
10 cars['car_company'] = cars['car_company'].replace(corrections)
11 cars.head()
```

✓ 0.0s

- The features `doornumber` and `cylindernumber` have words instead of numbers. We should convert this into numbers as it is easier to plot for our model.

```
1 # doornumber and cylindernumber have words instead of numbers
2
3 def wordToNum(word):
4     dict = {
5         'two': 2, 'three': 3, 'four': 4, 'five': 5,
6         'six': 6, 'eight': 8, 'twelve': 12
7     }
8     return dict.get(word, np.nan)
9
10 cars['doornumber'] = cars['doornumber'].apply(wordToNum)
11 cars['cylindernumber'] = cars['cylindernumber'].apply(wordToNum)
12
13 cars.head()
```

✓ 0.0s

- We then try to convert every column to a numerical one.

```
1 # try to convert every column to numerical
2
3 for col in cars.columns:
4     if cars[col].dtype == 'object':
5         try:
6             cars[col] = pd.to_numeric(cars[col])
7         except ValueError:
8             continue
```

✓ 0.0s

- If there are missing or null values, we can fill them with the median.

```
1 # fill missing values with median
2
3 for col in cars.select_dtypes(include=np.number).columns:
4     cars[col].fillna(cars[col].median(), inplace=True)
✓ 0.0s
```

- The car model name isn't necessary for regression as we already have the car company column. Thus, we can drop the `car_ID` and `CarName` columns.

```
1 # car model name isn't necessary for regression as we already have car company
2
3 cars = cars.drop(['car_ID', 'CarName'], axis=1)
4 cars.head()
✓ 0.0s
```

After cleaning the dataset, here is the info.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 205 entries, 0 to 204
Data columns (total 25 columns):
#   Column                Non-Null Count  Dtype
---  -
0   symboling              205 non-null   int64
1   fueltype               205 non-null   object
2   aspiration              205 non-null   object
3   doornumber             205 non-null   int64
4   carbody                205 non-null   object
5   drivewheel             205 non-null   object
6   enginelocation         205 non-null   object
7   wheelbase              205 non-null   float64
8   carlength              205 non-null   float64
9   carwidth               205 non-null   float64
10  carheight              205 non-null   float64
11  curbweight             205 non-null   int64
12  enginetype             205 non-null   object
13  cylindernumber         205 non-null   int64
14  enginesize             205 non-null   int64
15  fuelsystem             205 non-null   object
16  boreratio              205 non-null   float64
17  stroke                 205 non-null   float64
18  compressionratio       205 non-null   float64
19  horsepower             205 non-null   int64
...
23  price                  205 non-null   float64
24  car_company            205 non-null   object
dtypes: float64(8), int64(9), object(8)
memory usage: 40.2+ KB
```


2.2.3 One-hot encoding

We can convert our categorical columns into numerical columns using the process called one-hot encoding.

```
1 # one-hot encoding
2
3 features_with_dummies = pd.get_dummies(cars, columns=cars.select_dtypes(include=['object']).columns, drop_first=True)
✓ 0.0s Python
```

- `pd.get_dummies()` is the function which performs the conversion.
- `features` is the input DataFrame containing all the features.
- `columns=categorical_features` tells the function to only transform categorical features.
- `drop_first=True` creates a new column for all categories for each categorical column and drops the first one. This dropped category becomes the baseline and prevents multicollinearity.

2.3 Training the regression model

2.3.1 Manual train-test split

```
1 # manual train-test split
2
3 shuffled_df = features_with_dummies.sample(frac=1, random_state=69)
4
5 # split point at 0.7 (70%) for training
6 train_size = int(0.7 * len(shuffled_df))
7 train_df = shuffled_df[:train_size]
8 test_df = shuffled_df[train_size:]
9
10 y_train_raw = train_df.pop('price').values
11 X_train_df = train_df
12
13 y_test_raw = test_df.pop('price').values
14 X_test_df = test_df
✓ 0.0s
```

- `shuffled_df` randomly shuffles the entire dataset (`features_with_dummies`) before splitting.
 - `sample(frac=1)`: returns 100% of the rows but in a random order.
 - `random_state=69`: ensures the shuffle is reproducible.
- `train_size` calculates how many rows (samples) should go into the training set.
 - `len(shuffled_df)`: total number of samples
 - `0.7 * len(shuffled_df)`: takes 70% of the samples
 - `int(0.7 * len(shuffled_df))`: typecasts the output into an integer
- `train_df` selects the first 70% of the samples for the training set.
- `test_df` selects the remaining 30% for the testing set.

- `y_train_raw` extracts the target variable (`price`) from the training set.
 - `.pop('price')`: removes the 'price' column from `train_df` and returns it as a separate Series.
 - `.values`: converts that Series into a NumPy array.
- The remaining columns in `train_df` go to `X_train_df` as a feature matrix.
- `y_test_raw` does the same thing as `y_train` by excluding `price` but from the testing set instead of the training set.
- The remaining columns in `test_df` go to `X_test_df` as a feature matrix.

2.3.2 Feature scaling

```

1  # feature scaling
2
3  mean_X = X_train_df.mean()
4  std_X = X_train_df.std()
5  X_train_scaled = (X_train_df - mean_X) / std_X
6  X_test_scaled = (X_test_df - mean_X) / std_X
7
8  mean_y = y_train_raw.mean()
9  std_y = y_train_raw.std()
10 y_train = (y_train_raw - mean_y) / std_y
11 y_test = (y_test_raw - mean_y) / std_y
✓ 0.0s

```

- `mean_X` and `std_X` calculate the mean and standard deviation respectively for each feature in `X_train_df`.
- Z-score normalization is used to scale the features. The formula is:

$$Z = \frac{x - \mu}{\sigma}$$
- `X_train` and `X_test` are scaled to `X_train_scaled` and `X_test_scaled` respectively using this normalization.
- The same process is applied for `y_train_raw` and `y_test_raw`.

2.3.3 Gradient descent

```
1 # gradient descent
2
3 X_train = np.c_[np.ones(X_train_scaled.shape[0]), X_train_scaled.values]
4 X_test = np.c_[np.ones(X_test_scaled.shape[0]), X_test_scaled.values]
5
6 nan_cols = np.any(np.isnan(X_train), axis=0)
7 X_train = X_train[:, ~nan_cols]
8 X_test = X_test[:, ~nan_cols]
9
10 alpha = 0.05
11 iterations = 5000
12 m = len(y_train)
13 gradient_threshold = 1
14
15 theta = np.zeros(X_train.shape[1])
16 prev_cost = float('inf')
17 costs = []
18
19 for i in range(iterations):
20     predictions = X_train.dot(theta)
21     err = predictions - y_train
22     cost = np.mean(err ** 2)
23     costs.append(cost)
24
25     if abs(prev_cost - cost) < 1e-10:
26         print(f"Converged at iteration {i}")
27         break
28
29     gradient = (1/m) * X_train.T.dot(err)
30
31     gradient_norm = np.linalg.norm(gradient)
32     if gradient_norm > gradient_threshold:
33         gradient = gradient * gradient_threshold / gradient_norm
34
35     theta = theta - (alpha * gradient)
36     prev_cost = cost
```

✓ 0.0s

- A bias column is added to `X_train`.
 - `np.ones(...)` creates a column vector of ones with the same number of rows as `X_train_scaled`.
 - `np.c_` is a NumPy function that concatenates arrays column-wise.
 - This is done so that the model can learn a baseline value θ_0 that is independent of the input features. The equation becomes

$$y = \theta_0 + \theta_1 x_1 + \dots$$

- The exact same thing is done for `X_test`.
- The columns containing nan (not a number) or missing values are identified in the `nan_columns` variable.
- The `nan_columns` are then removed from `X_train` and `X_test`.
- The hyperparameters for gradient descent are set up.
 - Learning rate (`alpha`) = 0.05.
 - `iterations` = 5000
 - Total number of training samples (`m`).
 - Threshold for gradient clipping (`gradient_threshold`).
- Weights or coefficients (`theta`): initialized as a vector of zeros.

- Previous cost (`prev_cost`) is made to keep track of the cost of the previous iteration. It is initially set to infinity to make sure the convergence check does not trigger on the very first loop.
- A list of `costs` is created to store the cost at each iteration.
- The loop for gradient descent is set up.
 - The `predictions` variable stores the current predictions of the model on the training data. This is done by taking the dot product of `X_train` and `theta`.
 - The `err` variable stores the error (or residual) for each training example. It is calculated by subtracting the true `y_train` values from the `predictions`.
 - The `cost` variable calculates the cost function, which is MSE (mean squared error) in this case. Gradient descent is supposed to minimise this value.
 - The `cost` value at each iteration is appended to the `costs` list.
 - Convergence check: if the cost improves by a very tiny amount (less than $1e-10$), the model assumes that the cost has minimized and it stops.
 - The `gradient` variable calculates the gradient of the cost function. The formula is

$$\text{Gradient}(J(\theta)) = \frac{1}{m}X^T(X\theta - y)$$
 - The `gradient_norm` variable calculates the magnitude (L2 norm) of the gradient vector.
 - If the `gradient_norm` is greater than the `gradient_threshold`, it is clipped by scaling it down to a magnitude of 1.
 - Finally, `theta` is updated by subtracting it.
 - The `prev_cost` is updated to `cost`.

2.3.4 Calculating R^2

```

1 # calculate R-squared
2
3 y_pred = X_test.dot(theta)
4 ss_residuals = np.sum((y_test - y_pred)**2)
5 ss_total = np.sum((y_test - np.mean(y_test))**2)
6 r_squared = 1 - (ss_residuals / ss_total)
7
8 print(f"R-squared: {r_squared:.4f}")

```

✓ 0.0s

- `y_pred` are the model's predictions on the test data (`x_test`), using the final optimized parameters (`theta`) that were learnt using gradient descent.
- `ss_residuals` stands for the sum of squared residuals (SSR).
 - The residual can be calculated by subtracting `y_pred` from `y_test`.
 - Each residual is then squared and summed together by using `np.sum()`.
- `ss_total` stands for the total sum of squares (SST).
 - `y_test - np.mean(y_test)` finds the difference between each actual value and the overall mean of all actual values.
 - These are also squared and summed by using `np.sum()`.
- Finally, R^2 (`r_squared`) is calculated by the formula $R^2 = 1 - (SSR / SST)$ and printed.
- For our model, the R^2 value is 0.92. This means that our model explains approximately 92% of the variation in the target variable (`price`).

2.3.5 Plotting the cost vs iterations graph

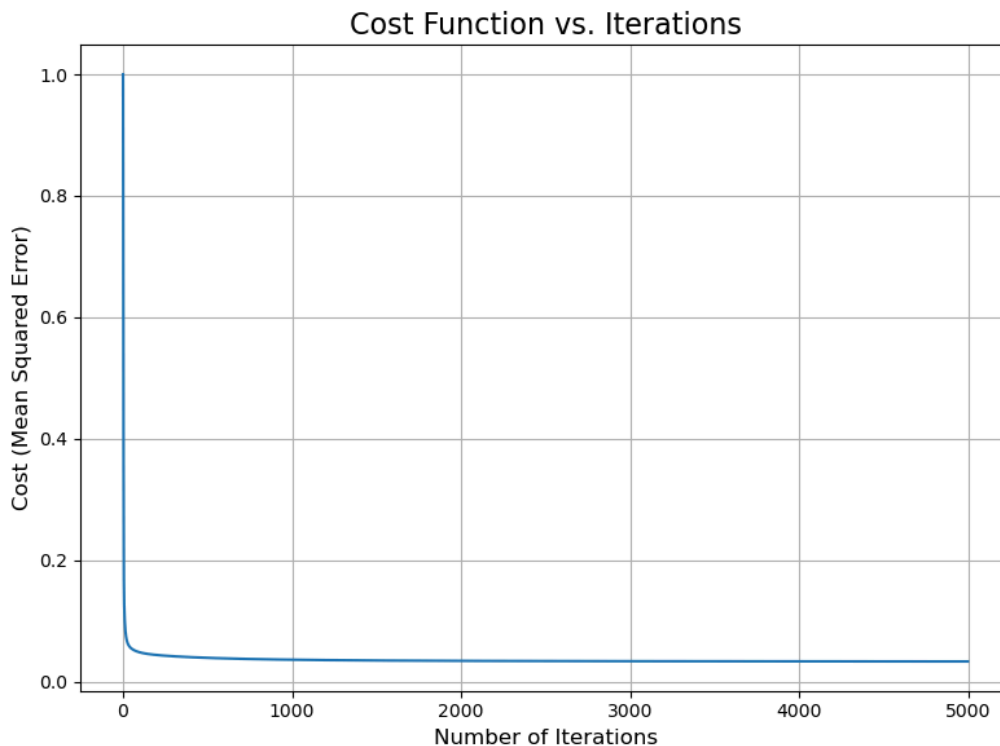
```

1  # cost vs iterations plot
2
3  plt.figure(figsize=(8, 6))
4  plt.plot(range(iterations), costs)
5  plt.title('Cost Function vs. Iterations', fontsize=16)
6  plt.xlabel('Number of Iterations', fontsize=12)
7  plt.ylabel('Cost (Mean Squared Error)', fontsize=12)
8  plt.grid(True)
9  plt.tight_layout()
✓ 0.0s

```

- The x-axis value is set to the range for `iterations`. This value goes from 0 to 5000 as we set 5000 iterations in gradient descent.
- The y-axis value is set to the `costs` list which was updated by the gradient descent.
- The `title`, `xlabel`, `ylabel` are given and a `grid` is added in the background to improve readability of the graph.
- `plt.tight_layout()` is used to ensure correct spacing of elements without overlap.

Analysis of the graph



- ***Rapid initial learning:*** In the beginning (roughly the first 1000 iterations), the cost drops very steeply. This shows that the model, starting from a random guess, was able to quickly find the general "downhill" direction and rapidly reduce its prediction errors.
- ***Convergence:*** After the initial drop, the curve gradually becomes flatter and plateaus. This is the most important part as it shows that the model has converged. This means it has found the point of minimum error (the "bottom of the valley") and further training iterations are not making any significant improvements.

This graph tells us that the hyperparameters of the learning rate (α) and the iterations were set well. The model learned what it needed to and then correctly stopped making major changes, resulting in a stable and optimized set of coefficients (θ).

2.3.6 Plotting the regression graph

```
1 # regression plot
2
3 plt.figure(figsize=(8, 6))
4 plt.scatter(y_test, y_pred, alpha=0.6)
5
6 line_coords = np.linspace(min(y_test.min(), y_pred.min()), max(y_test.max(), y_pred.max()), 100)
7 plt.plot(line_coords, line_coords, color='red', linestyle='--')
8
9 plt.title('Actual Prices vs. Predicted Prices', fontsize=16)
10 plt.xlabel('Actual Price ($)', fontsize=12)
11 plt.ylabel('Predicted Price ($)', fontsize=12)
12 plt.grid(True)
13 plt.tight_layout()
✓ 0.0s
```

- A scatter plot is created.
 - The x-axis represents the actual values (`y_test`).
 - The y-axis represents the predicted values (`y_pred`).
 - `alpha=0.6` makes the points slightly transparent, which makes them easier to see if they overlap.
- The `line_coords` variable prepares the coordinates for a reference line. It finds the absolute minimum and maximum values across both the actual and predicted data and then creates 100 evenly spaced points between them.
- A red dashed (--) diagonal line is plotted.
 - On this line, the x-value is equal to the y-value ($y = x$). It represents a perfect prediction.
 - If a scatter point falls on this line, it means the model's prediction was exactly correct.
- The `title`, `xlabel`, `ylabel` are given and a `grid` is added in the background to improve readability of the graph.
- `plt.tight_layout()` is used to ensure correct spacing of elements without overlap.

Analysis of the graph



- **High accuracy:** The blue points (representing individual cars) are very tightly clustered around the red dashed line. This red line represents a perfect prediction (where actual price = predicted price). Because the points are so close to this line, it means the model's predictions are very close to the true prices.
- **Strong positive correlation:** The clear, upward-sloping linear pattern of the points demonstrates a very strong positive relationship between the actual and predicted values. As the actual price of a car increases, the model's predicted price increases with it, which is ideal.
- **Visual proof of R-squared:** This graph is the visual representation of the high R-squared value which was calculated earlier. It shows what a model that can explain over 92% of the variance in prices looks like in practice.